





# Sumário

- 1 Lista
- 2 Lista Encadeada
- 3 Lista Circular
- 4 Listas Duplamente Encadeadas

# Listas

- Para armazenar um conjunto de elementos, estudamos o vetor.

```
1 #define MAX 1000
2 int vet[MAX];
```

```
1 int n;
2 scanf("%d", &n);
3 int* vet = (int*) malloc(n*sizeof(int));
```

- O vetor não é uma estrutura de dados muito flexível, pois precisamos dimensioná-lo com um número máximo de elementos.
  - Se o número de elementos exceder a dimensão do vetor, não existe uma maneira barata (computacionalmente) para alterarmos a dimensão.
  - Se o número de elementos for muito inferior à sua dimensão, estaremos subutilizando o espaço de memória.

# Listas

- Solução: usar estruturas de dados que cresçam à medida que precisarmos armazenar novos elementos e diminuam à medida que precisarmos retirar elementos armazenados anteriormente
  - Lista
  - Pilhas
  - Filas









# Listas

- O nó da lista pode ser representado pela estrutura abaixo

```
1 struct no {  
2     int info;  
3     struct no* prox;  
4 };  
5 typedef struct no No;
```

- A lista é representada pelo ponteiro para inicio e quantidade n

```
1 struct lista {  
2     No* inicio;  
3     int n;  
4 };  
5 typedef struct lista Lista;
```











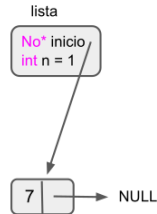


# Listas

- A função **insere** inclui um novo elemento:
    - Alocar dinamicamente;
    - O elemento é encadeado no início da lista.
- insere(lista, 7);

```

1 void insere(Lista* lista, int i) {
2     No* novo = (No*) malloc(sizeof(No));
3     novo->info = i;
4     novo->prox = lista->inicio;
5     lista->n++;
6     lista->inicio = novo;
7 }
    
```





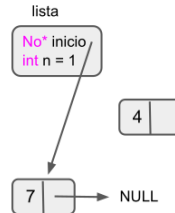




# Listas

- A função **insere** inclui um novo elemento:
    - Alocar dinamicamente;
    - O elemento é encadeado no início da lista.
- `insere(lista, 4);`

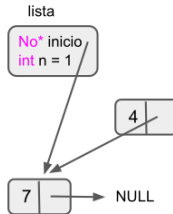
```
1 void insere(Lista* lista, int i) {  
2     No* novo = (No*) malloc(sizeof(No));  
3     novo->info = i;   
4     novo->prox = lista->inicio;  
5     lista->n++;  
6     lista->inicio = novo;  
7 }
```



# Listas

- A função **insere** inclui um novo elemento:
    - Alocar dinamicamente;
    - O elemento é encadeado no início da lista.
- insere(lista, 4);

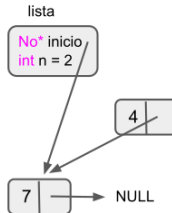
```
1 void insere(Lista* lista, int i) {  
2     No* novo = (No*) malloc(sizeof(No));  
3     novo->info = i;  
4     novo->prox = lista->inicio; ←  
5     lista->n++;  
6     lista->inicio = novo;  
7 }
```



# Listas

- A função **insere** inclui um novo elemento:
    - Alocar dinamicamente;
    - O elemento é encadeado no início da lista.
- insere(lista, 4);

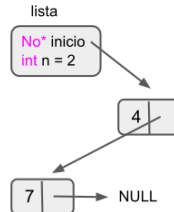
```
1 void insere(Lista* lista, int i) {  
2     No* novo = (No*) malloc(sizeof(No));  
3     novo->info = i;  
4     novo->prox = lista->inicio;  
5     lista->n++;  
6     lista->inicio = novo;  
7 }
```



# Listas

- A função **insere** inclui um novo elemento:
    - Alocar dinamicamente;
    - O elemento é encadeado no início da lista.
- insere(lista, 4);

```
1 void insere(Lista* lista, int i) {  
2     No* novo = (No*) malloc(sizeof(No));  
3     novo->info = i;  
4     novo->prox = lista->inicio;  
5     lista->n++;  
6     lista->inicio = novo; ◀  
7 }
```



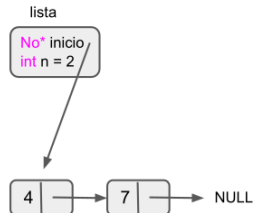
# Listas

- A função **insere** inclui um novo elemento:
    - Alocar dinamicamente;
    - O elemento é encadeado no início da lista.
- ```
insere(lista, 4);
```

```

1 void insere(Lista* lista, int i) {
2     No* novo = (No*) malloc(sizeof(No)),
3     novo->info = i;
4     novo->prox = lista->inicio;
5     lista->n++;
6     lista->inicio = novo; ←
7 }

```



# Listas

- A função **insere** inclui um novo elemento:
  - Alocar dinamicamente;
  - O elemento é encadeado no início da lista.

```
1 int main() {
2     Lista* lista;
3     lista = inicializa();
4     insere(lista, 10);
5     insere(lista, 20);
6     insere(lista, 30);
7     return 0;
8 }
```



# Listas

- A função **imprime** percorre todos os elementos:

```
1 void imprime(Lista* lista) {
2     No* p = lista->inicio;
3     while(p != NULL) {
4         printf("%d, ", p->info);
5         p = p->prox;
6     }
7     printf("\n");
8 }
```

# Listas

- A função **busca** percorre todos os elementos e retorna o elemento buscado caso ele exista.

```
1 No* busca(Lista* lista, int v) {  
2     No* p = lista->inicio;  
3     while(p != NULL) {  
4         if(p->info == v) {  
5             return p;  
6         }  
7         p = p->prox;  
8     }  
9     return NULL;  
10 }
```

```
1 int main() {  
2     Lista* lista;  
3     lista = inicializa();  
4     insere(lista, 9);  
5     insere(lista, 6);  
6  
7     No* no = busca(lista, 9);  
8     if(no != NULL) {  
9         printf("Encontrou %d", no->info);  
10    }  
11  
12    return 0;  
13 }
```

# Listas

- Como ficaria a busca se o retorno for a posição na lista?

# Listas

- Como ficaria a busca se o retorno for a posição na lista?

```

1 int busca_pos(Lista* lista, int v) {
2     No* p = lista->inicio;
3     int pos = 0;
4     while(p != NULL) {
5         if(p->info == v) {
6             return pos;
7         }
8         p = p->prox;
9         pos++;
10    }
11    return -1;
12 }

```

# Listas

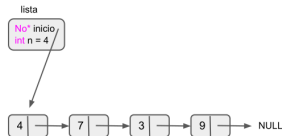
- A função **retira** remove um elemento específico.

```

1 void retira(Lista* lista, int v) {
2     No* p = lista->inicio;
3     No* ant = NULL;
4     while(p != NULL) {
5         if(p->info == v) break;
6         ant = p;
7         p = p->prox;
8     }
9     if(p == NULL) return;
10    if(ant == NULL) lista->inicio = p->prox;
11    else ant->prox = p->prox;
12    free(p);
13    lista->n--;
14 }

```

```
retira(lista, 7);
```

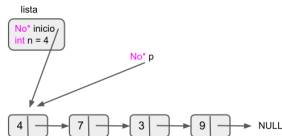


# Listas

- A função **retira** remove um elemento específico.

```
1 void retira(Lista* lista, int v) {  
2     No* p = lista->inicio; ←  
3     No* ant = NULL;  
4     while(p != NULL) {  
5         if(p->info == v) break;  
6         ant = p;  
7         p = p->prox;  
8     }  
9     if(p == NULL) return;  
10    if(ant == NULL) lista->inicio = p->prox;  
11    else ant->prox = p->prox;  
12    free(p);  
13    lista->n--;  
14 }
```

retira(lista, 7);

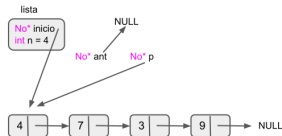


# Listas

- A função **retira** remove um elemento específico.

```
1 void retira(Lista* lista, int v) {  
2     No* p = lista->inicio;  
3     No* ant = NULL; ←  
4     while(p != NULL) {  
5         if(p->info == v) break;  
6         ant = p;  
7         p = p->prox;  
8     }  
9     if(p == NULL) return;  
10    if(ant == NULL) lista->inicio = p->prox;  
11    else ant->prox = p->prox;  
12    free(p);  
13    lista->n--;  
14 }
```

retira(lista, 7);



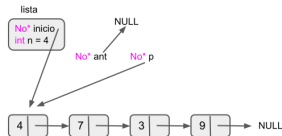
# Listas

- A função **retira** remove um elemento específico.

```

1 void retira(Lista* lista, int v) {
2     No* p = lista->inicio;
3     No* ant = NULL;
4     while(p != NULL) { ←
5         if(p->info == v) break;
6         ant = p;
7         p = p->prox;
8     }
9     if(p == NULL) return;
10    if(ant == NULL) lista->inicio = p->prox;
11    else ant->prox = p->prox;
12    free(p);
13    lista->n--;
14 }
    
```

retira(lista, 7);



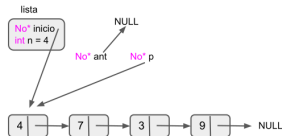


# Listas

- A função **retira** remove um elemento específico.

```
1 void retira(Lista* lista, int v) {  
2     No* p = lista->inicio;  
3     No* ant = NULL;  
4     while(p != NULL) {  
5         if(p->info == v) break; ←  
6         ant = p;  
7         p = p->prox;  
8     }  
9     if(p == NULL) return;  
10    if(ant == NULL) lista->inicio = p->prox;  
11    else ant->prox = p->prox;  
12    free(p);  
13    lista->n--;  
14 }
```

retira(lista, 7);

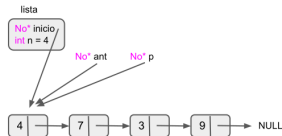


# Listas

- A função **retira** remove um elemento específico.

```
1 void retira(Lista* lista, int v) {  
2     No* p = lista->inicio;  
3     No* ant = NULL;  
4     while(p != NULL) {  
5         if(p->info == v) break;  
6         ant = p; ←  
7         p = p->prox;  
8     }  
9     if(p == NULL) return;  
10    if(ant == NULL) lista->inicio = p->prox;  
11    else ant->prox = p->prox;  
12    free(p);  
13    lista->n--;  
14 }
```

retira(lista, 7);



# Listas

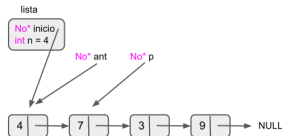
- A função **retira** remove um elemento específico.

```

1 void retira(Lista* lista, int v) {
2     No* p = lista->inicio;
3     No* ant = NULL;
4     while(p != NULL) {
5         if(p->info == v) break;
6         ant = p;
7         p = p->prox;
8     }
9     if(p == NULL) return;
10    if(ant == NULL) lista->inicio = p->prox;
11    else ant->prox = p->prox;
12    free(p);
13    lista->n--;
14 }

```

retira(lista, 7);



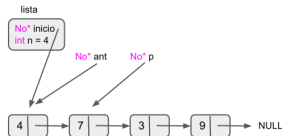


# Listas

- A função **retira** remove um elemento específico.

```
1 void retira(Lista* lista, int v) {  
2     No* p = lista->inicio;  
3     No* ant = NULL;  
4     while(p != NULL) {  
5         if(p->info == v) break; ←  
6         ant = p;  
7         p = p->prox;  
8     }  
9     if(p == NULL) return;  
10    if(ant == NULL) lista->inicio = p->prox;  
11    else ant->prox = p->prox;  
12    free(p);  
13    lista->n--;  
14 }
```

retira(lista, 7);

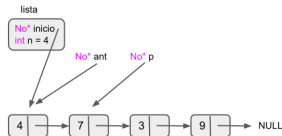


# Listas

- A função **retira** remove um elemento específico.

```
1 void retira(Lista* lista, int v) {  
2     No* p = lista->inicio;  
3     No* ant = NULL;  
4     while(p != NULL) {  
5         if(p->info == v) break;  
6         ant = p;  
7         p = p->prox;  
8     }  
9     if(p == NULL) return; ⇐  
10    if(ant == NULL) lista->inicio = p->prox;  
11    else ant->prox = p->prox;  
12    free(p);  
13    lista->n--;  
14 }
```

retira(lista, 7);

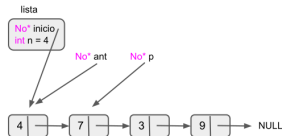


# Listas

- A função **retira** remove um elemento específico.

```
1 void retira(Lista* lista, int v) {  
2     No* p = lista->inicio;  
3     No* ant = NULL;  
4     while(p != NULL) {  
5         if(p->info == v) break;  
6         ant = p;  
7         p = p->prox;  
8     }  
9     if(p == NULL) return;  
10    if(ant == NULL) lista->inicio = p->prox; ←  
11    else ant->prox = p->prox;  
12    free(p);  
13    lista->n--;  
14 }
```

retira(lista, 7);



# Listas

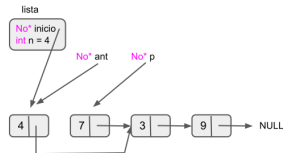
- A função **retira** remove um elemento específico.

```

1 void retirar(Lista* lista, int v) {
2     No* p = lista->inicio;
3     No* ant = NULL;
4     while(p != NULL) {
5         if(p->info == v) break;
6         ant = p;
7         p = p->prox;
8     }
9     if(p == NULL) return;
10    if(ant == NULL) lista->inicio = p->prox;
11    else ant->prox = p->prox;
12    free(p);
13    lista->n--;
14 }

```

```
retira(lista, 7);
```





# Listas

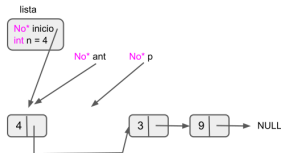
- A função **retira** remove um elemento específico.

```

1 void retira(Lista* lista, int v) {
2     No* p = lista->inicio;
3     No* ant = NULL;
4     while(p != NULL) {
5         if(p->info == v) break;
6         ant = p;
7         p = p->prox;
8     }
9     if(p == NULL) return;
10    if(ant == NULL) lista->inicio = p->prox;
11    else ant->prox = p->prox;
12    free(p);
13    lista->n--;
14 }

```

```
retira(lista, 7);
```

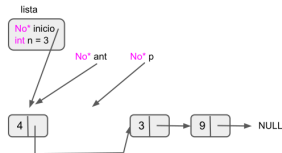


# Listas

- A função **retira** remove um elemento específico.

```
1 void retira(Lista* lista, int v) {  
2     No* p = lista->inicio;  
3     No* ant = NULL;  
4     while(p != NULL) {  
5         if(p->info == v) break;  
6         ant = p;  
7         p = p->prox;  
8     }  
9     if(p == NULL) return;  
10    if(ant == NULL) lista->inicio = p->prox;  
11    else ant->prox = p->prox;  
12    free(p);  
13    lista->n--; ←  
14 }
```

retira(lista, 7);

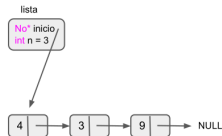


# Listas

- A função **retira** remove um elemento específico.

```
1 void retira(Lista* lista, int v) {  
2     No* p = lista->inicio;  
3     No* ant = NULL;  
4     while(p != NULL) {  
5         if(p->info == v) break;  
6         ant = p;  
7         p = p->prox;  
8     }  
9     if(p == NULL) return;  
10    if(ant == NULL) lista->inicio = p->prox;  
11    else ant->prox = p->prox;  
12    free(p);  
13    lista->n--;  
14 }
```

retira(lista, 7);



# Listas

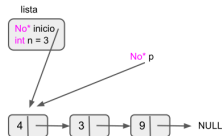
- A função **retira** remove um elemento específico.

```

1 void retira(Lista* lista, int v) {
2     No* p = lista->inicio; ←
3     No* ant = NULL;
4     while(p != NULL) {
5         if(p->info == v) break;
6         ant = p;
7         p = p->prox;
8     }
9     if(p == NULL) return;
10    if(ant == NULL) lista->inicio = p->prox;
11    else ant->prox = p->prox;
12    free(p);
13    lista->n--;
14 }

```

```
retira(lista, 4);
```

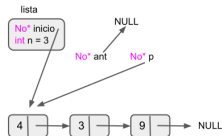


# Listas

- A função **retira** remove um elemento específico.

```
1 void retira(Lista* lista, int v) {  
2     No* p = lista->inicio;  
3     No* ant = NULL; ←  
4     while(p != NULL) {  
5         if(p->info == v) break;  
6         ant = p;  
7         p = p->prox;  
8     }  
9     if(p == NULL) return;  
10    if(ant == NULL) lista->inicio = p->prox;  
11    else ant->prox = p->prox;  
12    free(p);  
13    lista->n--;  
14 }
```

retira(lista, 4);

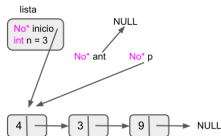


# Listas

- A função **retira** remove um elemento específico.

```
1 void retira(Lista* lista, int v) {  
2     No* p = lista->inicio;  
3     No* ant = NULL;  
4     while(p != NULL) { ←  
5         if(p->info == v) break;  
6         ant = p;  
7         p = p->prox;  
8     }  
9     if(p == NULL) return;  
10    if(ant == NULL) lista->inicio = p->prox;  
11    else ant->prox = p->prox;  
12    free(p);  
13    lista->n--;  
14 }
```

retira(lista, 4);

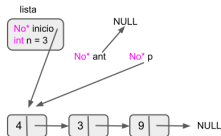


# Listas

- A função **retira** remove um elemento específico.

```
1 void retira(Lista* lista, int v) {  
2     No* p = lista->inicio;  
3     No* ant = NULL;  
4     while(p != NULL) {  
5         if(p->info == v) break; ←  
6         ant = p;  
7         p = p->prox;  
8     }  
9     if(p == NULL) return;  
10    if(ant == NULL) lista->inicio = p->prox;  
11    else ant->prox = p->prox;  
12    free(p);  
13    lista->n--;  
14 }
```

retira(lista, 4);

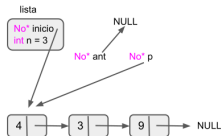


# Listas

- A função **retira** remove um elemento específico.

```
1 void retira(Lista* lista, int v) {  
2     No* p = lista->inicio;  
3     No* ant = NULL;  
4     while(p != NULL) {  
5         if(p->info == v) break;  
6         ant = p;  
7         p = p->prox;  
8     }  
9     if(p == NULL) return; ⇐  
10    if(ant == NULL) lista->inicio = p->prox;  
11    else ant->prox = p->prox;  
12    free(p);  
13    lista->n--;  
14 }
```

retira(lista, 4);



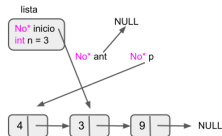


# Listas

- A função **retira** remove um elemento específico.

```
1 void retira(Lista* lista, int v) {  
2     No* p = lista->inicio;  
3     No* ant = NULL;  
4     while(p != NULL) {  
5         if(p->info == v) break;  
6         ant = p;  
7         p = p->prox;  
8     }  
9     if(p == NULL) return;  
10    if(ant == NULL) lista->inicio = p->prox; ⇐  
11    else ant->prox = p->prox;  
12    free(p);  
13    lista->n--;  
14 }
```

retira(lista, 4);



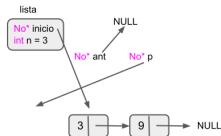
# Listas

- A função **retira** remove um elemento específico.

```

1 void retira(Lista* lista, int v) {
2     No* p = lista->inicio;
3     No* ant = NULL;
4     while(p != NULL) {
5         if(p->info == v) break;
6         ant = p;
7         p = p->prox;
8     }
9     if(p == NULL) return;
10    if(ant == NULL) lista->inicio = p->prox;
11    else ant->prox = p->prox;
12    free(p);
13    lista->n--;
14 }
    
```

retira(lista, 4);



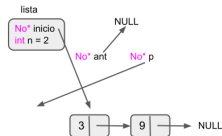
# Listas

- A função **retira** remove um elemento específico.

```

1 void retira(Lista* lista, int v) {
2     No* p = lista->inicio;
3     No* ant = NULL;
4     while(p != NULL) {
5         if(p->info == v) break;
6         ant = p;
7         p = p->prox;
8     }
9     if(p == NULL) return;
10    if(ant == NULL) lista->inicio = p->prox;
11    else ant->prox = p->prox;
12    free(p);
13    lista->n--;
14 }
    
```

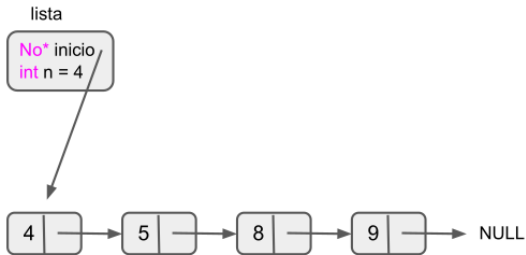
retira(lista, 4);





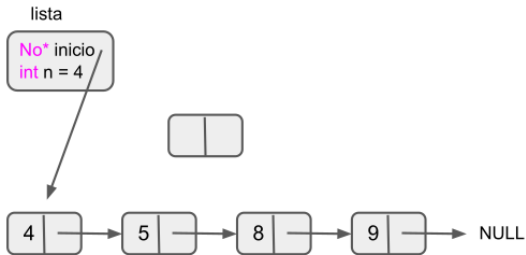
# Listas

- Inserção em posições diferentes do início.
- Interessante para manter, por exemplo, a lista ordenada.



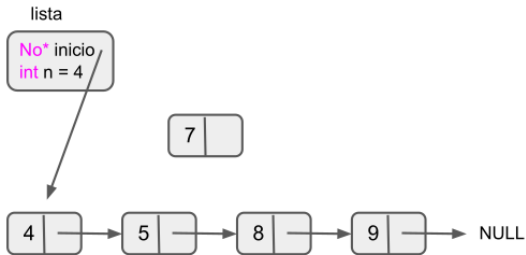
# Listas

- Inserção em posições diferentes do início.
- Interessante para manter, por exemplo, a lista ordenada.



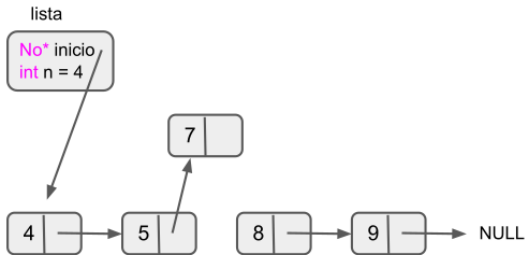
# Listas

- Inserção em posições diferentes do início.
- Interessante para manter, por exemplo, a lista ordenada.



# Listas

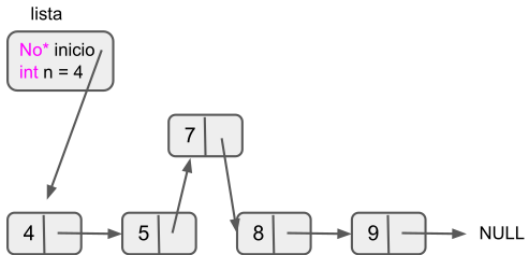
- Inserção em posições diferentes do início.
- Interessante para manter, por exemplo, a lista ordenada.





# Listas

- Inserção em posições diferentes do início.
- Interessante para manter, por exemplo, a lista ordenada.



# Listas

- Inserção em posições diferentes do início.
- Interessante para manter, por exemplo, a lista ordenada.

```
1 void insere_ordenado(Lista* lista, int v) {
2     No* ant = NULL;
3     No* p = lista->inicio;
4     No* novo = (No*) malloc(sizeof(No));
5     novo->info = v;
6     novo->prox = NULL;
7
8     /* procura posição de inserção */
9     while (p != NULL && p->info < v) {
10         ant = p;
11         p = p->prox;
12     }
13     if (ant == NULL) {
14         /* insere elemento no início */
15         novo->prox = lista->inicio;
16         lista->inicio = novo;
17     } else {
18         /* insere elemento no meio da lista */
19         novo->prox = ant->prox;
20         ant->prox = novo;
21     }
22 }
```

- Como ficaria a função para inserir um elemento em uma determinada posição?

# Listas

- Função para liberar uma lista.

```

1 void libera(Lista* lista) {
2     No* p = lista->inicio;
3     while(p != NULL) {
4         No* t = p->prox;
5         free(p);
6         p = t;
7     }
8     free(lista);
9 }
10

```

# Sumário

- 1 Lista
- 2 Lista Encadeada
- 3 Lista Circular
- 4 Listas Duplamente Encadeadas

# Lista Circular

- Numa lista circular, o último elemento tem como próximo o primeiro elemento da lista, formando um ciclo.

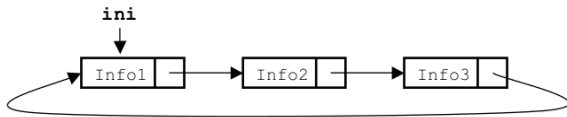


Figura: Lista circular.

# Lista Circular

- Numa lista circular, o último elemento tem como próximo o primeiro elemento da lista, formando um ciclo.

```

1 void imprime_circular(Lista* lista) {
2     No* p = lista->inicio;
3     /* testa se lista não é vazia */
4     if (p != NULL) {
5         /* percorre os elementos até alcançar novamente o início */
6         do {
7             printf("%d\n", p->info); /* imprime informação do nó */
8             p = p->prox; /* avança para o próximo nó */
9         } while (p != lista->inicio);
10    }
11 }

```

# Lista Circular

## Exercícios

Escreva as funções para inserir e retirar um elemento de uma lista circular.



# Sumário

- 1 Lista
- 2 Lista Encadeada
- 3 Lista Circular
- 4 Listas Duplamente Encadeadas



## Lista Duplamente Encadeada

- Lista duplamente encadeada:
  - Cada elemento tem um ponteiro para o próximo elemento e um ponteiro para o elemento anterior.



Figura: Lista duplamente encadeada.

# Lista Duplamente Encadeada

- O nó e a lista podem ser representados pelas estruturas abaixo

```
1 struct no {  
2     int info;  
3     struct no* prox;  
4     struct no* ant;  
5 };  
6 typedef struct no No;  
7  
8 struct lista {  
9     No* inicio;  
10    int n;  
11 };  
12 typedef struct lista Lista;
```

# Lista Duplamente Encadeada

- Inicialização:

```
1 Lista* inicializa() {  
2     Lista* lista = (Lista*) malloc(sizeof(Lista));  
3     lista->inicio = NULL;  
4     lista->n = 0;  
5     return lista;  
6 }  
7
```

# Lista Duplamente Encadeada

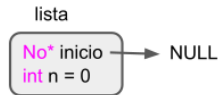
- Inserir no início da lista

```
insere(lista, 7);
```

```

1 void insere(Lista* lista, int v) {
2     No* novo = (No*) malloc(sizeof(No));
3     novo->info = v;
4     novo->prox = lista->inicio;
5     novo->ant = NULL;
6
7     if(lista->inicio != NULL) {
8         lista->inicio->ant = novo;
9     }
10    lista->n++;
11    lista->inicio = novo;
12 }

```

























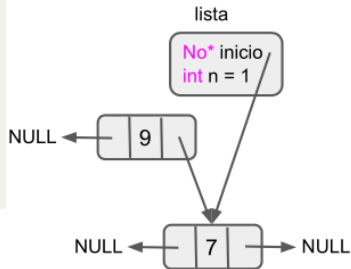


# Lista Duplamente Encadeada

- Inserir no início da lista

insere(lista, 9);

```
1 void insere(Lista* lista, int v) {  
2     No* novo = (No*) malloc(sizeof(No));  
3     novo->info = v;  
4     novo->prox = lista->inicio;  
5     novo->ant = NULL;   
6  
7     if(lista->inicio != NULL) {  
8         lista->inicio->ant = novo;  
9     }  
10    lista->n++;  
11    lista->inicio = novo;  
12 }
```













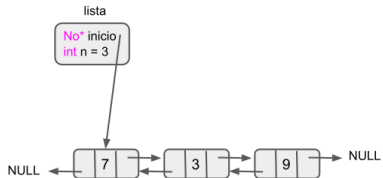
# Lista Duplamente Encadeada

## Remover um elemento.

```

1 void retira(Lista* lista, int v) {
2     No* p = busca(lista, v);
3     if(p == NULL) return;
4
5     // Remove o primeiro elemento
6     if(lista->inicio == p) {
7         lista->inicio = p->prox;
8     } else {
9         p->ant->prox = p->prox;
10    }
11    if(p->prox != NULL) {
12        p->prox->ant = p->ant;
13    }
14    free(p);
15    lista->n--;
16 }
    
```

retira(lista, 3);



# Lista Duplamente Encadeada

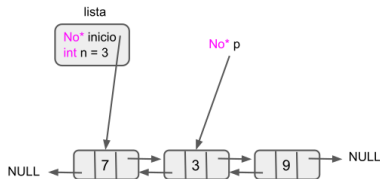
- Remover um elemento.

```

1 void retira(Lista* lista, int v) {
2     No* p = busca(lista, v); ←
3     if(p == NULL) return;
4
5     // Remove o primeiro elemento
6     if(lista->inicio == p) {
7         lista->inicio = p->prox;
8     } else {
9         p->ant->prox = p->prox;
10    }
11    if(p->prox != NULL) {
12        p->prox->ant = p->ant;
13    }
14    free(p);
15    lista->n--;
16 }

```

```
retira(lista, 3);
```





# Lista Duplamente Encadeada

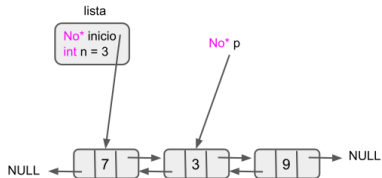
- Remover um elemento.

```

1 void retira(Lista* lista, int v) {
2     No* p = busca(lista, v);
3     if(p == NULL) return; ←
4
5     // Remove o primeiro elemento
6     if(lista->inicio == p) {
7         lista->inicio = p->prox;
8     } else {
9         p->ant->prox = p->prox;
10    }
11    if(p->prox != NULL) {
12        p->prox->ant = p->ant;
13    }
14    free(p);
15    lista->n--;
16 }

```

```
retira(lista, 3);
```



## Lista Duplamente Encadeada

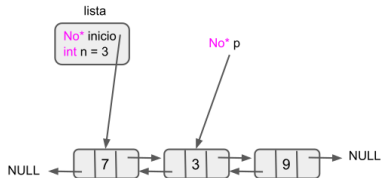
- Remover um elemento.

```

1 void retira(Lista* lista, int v) {
2     No* p = busca(lista, v);
3     if(p == NULL) return;
4
5     // Remove o primeiro elemento
6     if(lista->inicio == p) { ←
7         lista->inicio = p->prox;
8     } else {
9         p->ant->prox = p->prox;
10    }
11    if(p->prox != NULL) {
12        p->prox->ant = p->ant;
13    }
14    free(p);
15    lista->n--;
16 }

```

```
retira(lista, 3);
```



# Lista Duplamente Encadeada

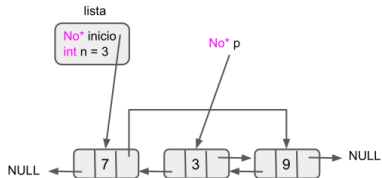
- Remover um elemento.

```

1 void retira(Lista* lista, int v) {
2     No* p = busca(lista, v);
3     if(p == NULL) return;
4
5     // Remove o primeiro elemento
6     if(lista->inicio == p) {
7         lista->inicio = p->prox;
8     } else {
9         p->ant->prox = p->prox; ←
10    }
11    if(p->prox != NULL) {
12        p->prox->ant = p->ant;
13    }
14    free(p);
15    lista->n--;
16 }

```

```
retira(lista, 3);
```



## Lista Duplamente Encadeada

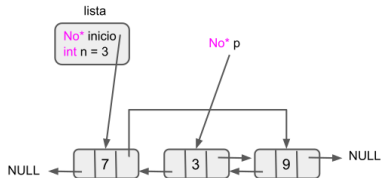
- Remover um elemento.

```

1 void retira(Lista* lista, int v) {
2     No* p = busca(lista, v);
3     if(p == NULL) return;
4
5     // Remove o primeiro elemento
6     if(lista->inicio == p) {
7         lista->inicio = p->prox;
8     } else {
9         p->ant->prox = p->prox;
10    }
11    if(p->prox != NULL) { ←
12        p->prox->ant = p->ant;
13    }
14    free(p);
15    lista->n--;
16 }

```

```
retira(lista, 3);
```



# Lista Duplamente Encadeada

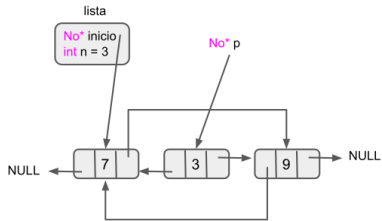
- Remover um elemento.

```

1 void retira(Lista* lista, int v) {
2     No* p = busca(lista, v);
3     if(p == NULL) return;
4
5     // Remove o primeiro elemento
6     if(lista->inicio == p) {
7         lista->inicio = p->prox;
8     } else {
9         p->ant->prox = p->prox;
10    }
11    if(p->prox != NULL) {
12        p->prox->ant = p->ant; ←
13    }
14    free(p);
15    lista->n--;
16 }

```

```
retira(lista, 3);
```



# Lista Duplamente Encadeada

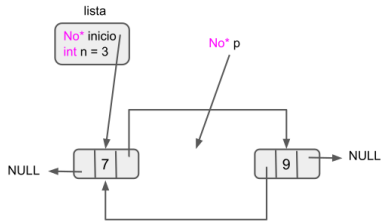
- Remover um elemento.

```

1 void retira(Lista* lista, int v) {
2     No* p = busca(lista, v);
3     if(p == NULL) return;
4
5     // Remove o primeiro elemento
6     if(lista->inicio == p) {
7         lista->inicio = p->prox;
8     } else {
9         p->ant->prox = p->prox;
10    }
11    if(p->prox != NULL) {
12        p->prox->ant = p->ant;
13    }
14    free(p);
15    lista->n--;
16 }

```

```
retira(lista, 3);
```



# Lista Duplamente Encadeada

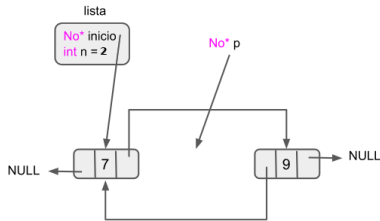
- Remover um elemento.

```

1 void retira(Lista* lista, int v) {
2     No* p = busca(lista, v);
3     if(p == NULL) return;
4
5     // Remove o primeiro elemento
6     if(lista->inicio == p) {
7         lista->inicio = p->prox;
8     } else {
9         p->ant->prox = p->prox;
10    }
11    if(p->prox != NULL) {
12        p->prox->ant = p->ant;
13    }
14    free(p);
15    lista->n--;
16 }

```

```
retira(lista, 3);
```



# Lista Duplamente Encadeada

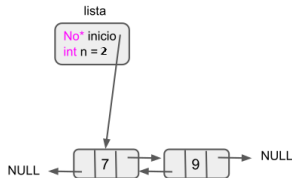
- Remover um elemento.

```

1 void retira(Lista* lista, int v) {
2     No* p = busca(lista, v);
3     if(p == NULL) return;
4
5     // Remove o primeiro elemento
6     if(lista->inicio == p) {
7         lista->inicio = p->prox;
8     } else {
9         p->ant->prox = p->prox;
10    }
11    if(p->prox != NULL) {
12        p->prox->ant = p->ant;
13    }
14    free(p);
15    lista->n--;
16 }

```

```
retira(lista, 7);
```





# Lista Duplamente Encadeada

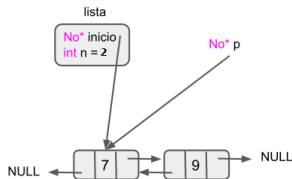
- Remover um elemento.

```

1 void retira(Lista* lista, int v) {
2     No* p = busca(lista, v); ←
3     if(p == NULL) return;
4
5     // Remove o primeiro elemento
6     if(lista->inicio == p) {
7         lista->inicio = p->prox;
8     } else {
9         p->ant->prox = p->prox;
10    }
11    if(p->prox != NULL) {
12        p->prox->ant = p->ant;
13    }
14    free(p);
15    lista->n--;
16 }

```

```
retira(lista, 7);
```



# Lista Duplamente Encadeada

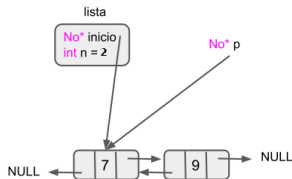
- Remover um elemento.

```

1 void retira(Lista* lista, int v) {
2     No* p = busca(lista, v);
3     if(p == NULL) return; ←
4
5     // Remove o primeiro elemento
6     if(lista->inicio == p) {
7         lista->inicio = p->prox;
8     } else {
9         p->ant->prox = p->prox;
10    }
11    if(p->prox != NULL) {
12        p->prox->ant = p->ant;
13    }
14    free(p);
15    lista->n--;
16 }

```

```
retira(lista, 7);
```



# Lista Duplamente Encadeada

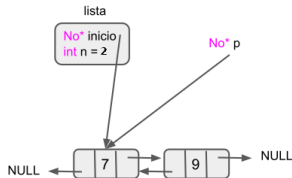
- Remover um elemento.

```

1 void retira(Lista* lista, int v) {
2     No* p = busca(lista, v);
3     if(p == NULL) return;
4
5     // Remove o primeiro elemento
6     if(lista->inicio == p) { ←
7         lista->inicio = p->prox;
8     } else {
9         p->ant->prox = p->prox;
10    }
11    if(p->prox != NULL) {
12        p->prox->ant = p->ant;
13    }
14    free(p);
15    lista->n--;
16 }

```

```
retira(lista, 7);
```



# Lista Duplamente Encadeada

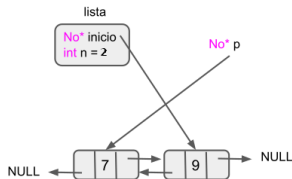
- Remover um elemento.

```

1 void retira(Lista* lista, int v) {
2     No* p = busca(lista, v);
3     if(p == NULL) return;
4
5     // Remove o primeiro elemento
6     if(lista->inicio == p) {
7         lista->inicio = p->prox; ←
8     } else {
9         p->ant->prox = p->prox;
10    }
11    if(p->prox != NULL) {
12        p->prox->ant = p->ant;
13    }
14    free(p);
15    lista->n--;
16 }

```

```
retira(lista, 7);
```



# Lista Duplamente Encadeada

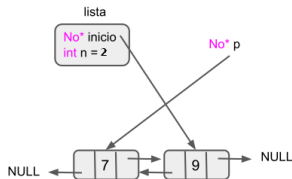
- Remover um elemento.

```

1 void retira(Lista* lista, int v) {
2     No* p = busca(lista, v);
3     if(p == NULL) return;
4
5     // Remove o primeiro elemento
6     if(lista->inicio == p) {
7         lista->inicio = p->prox;
8     } else {
9         p->ant->prox = p->prox;
10    }
11    if(p->prox != NULL) { ←
12        p->prox->ant = p->ant;
13    }
14    free(p);
15    lista->n--;
16 }

```

```
retira(lista, 7);
```



# Lista Duplamente Encadeada

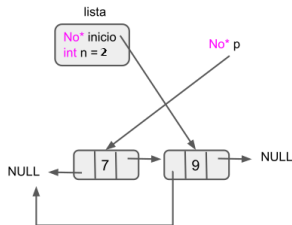
- Remover um elemento.

```

1 void retira(Lista* lista, int v) {
2     No* p = busca(lista, v);
3     if(p == NULL) return;
4
5     // Remove o primeiro elemento
6     if(lista->inicio == p) {
7         lista->inicio = p->prox;
8     } else {
9         p->ant->prox = p->prox;
10    }
11    if(p->prox != NULL) {
12        p->prox->ant = p->ant; ←
13    }
14    free(p);
15    lista->n--;
16 }

```

```
retira(lista, 7);
```



# Lista Duplamente Encadeada

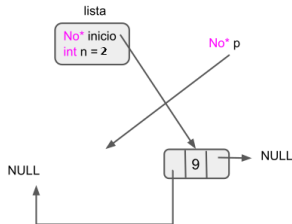
- Remover um elemento.

```

1 void retira(Lista* lista, int v) {
2     No* p = busca(lista, v);
3     if(p == NULL) return;
4
5     // Remove o primeiro elemento
6     if(lista->inicio == p) {
7         lista->inicio = p->prox;
8     } else {
9         p->ant->prox = p->prox;
10    }
11    if(p->prox != NULL) {
12        p->prox->ant = p->ant;
13    }
14    free(p);
15    lista->n--;
16 }

```

```
retira(lista, 7);
```



# Lista Duplamente Encadeada

- Remover um elemento.

```

1 void retira(Lista* lista, int v) {
2     No* p = busca(lista, v);
3     if(p == NULL) return;
4
5     // Remove o primeiro elemento
6     if(lista->inicio == p) {
7         lista->inicio = p->prox;
8     } else {
9         p->ant->prox = p->prox;
10    }
11    if(p->prox != NULL) {
12        p->prox->ant = p->ant;
13    }
14    free(p);
15    lista->n--;
16 }

```

```
retira(lista, 7);
```

